BRAINSTER

TypeScript

Brainster Web Development Academy

Table of contents

Interfaces

TypeScript - extending

```
interface Human {
    name: string,
    age: number
}
interface Person extends Human {
    gender: string
}
const obj:Person = {
    name: 'Riste',
    age: 34,
}
// Property 'gender' is missing in type '{
name: string; age: number; }' but required
in type 'Person'
```

TypeScript - merging

```
interface Animal {
    name: string,
    color: string
}
interface Animal {
    type: string
}
const dog: Animal = {
    name: 'Laika',
    color: 'Black and White',
}
// Property 'type' is missing in type '{
name: string; color: string; }' but required
in type 'Animal'.
```


Generics

TypeScript - extending

```
function removeItemFromArray<T>(arr: Array<T>, item: T) {
    const idx = arr.findIndex(el => item === el)
    arr.splice(idx, 1)
    return arr
}

removeItemFromArray([1, 20, 2, 3], 20)
removeItemFromArray(['1', '20', '2', '3'], '20')
```

- Generics act like variables of types, meaning whatever we pass when we execute the function that will be the type

- Useful to write more generic functions for greater reusability

Return function types

TypeScript

```
function add(n1: number, n2: number) {
    return n1 + n2
}
console.log(add(3, 4)) // returns number
```


- Functions that have return value in TypeScript also have inferred return type, meaning TypeScript will try to understand from your code what type of value is returned
- We should not, but we can specify the return type of the function using :[return type] after the parameter parentheses like this

TypeScript

```
function add(n1: number, n2: number): number {
    return n1 + n2
}
```

Return function types - void

TypeScript

```
function logResult(n:number){
    console.log('Result is: ', n)
}
```


- This example function does not contain return inside, therefore it return nothing (void)
- Not to get confused in JavaScript this kind of function return “undefined”, but TypeScript threats functions a bit differently

TypeScript

```
function logResult(n:number){
    console.log('Result is: ', n)
}

console.log(logResult(100)) // undefined
```

Function types

TypeScript

```
const add = (n1: number, n2: number) => n1 + n2
const logResult = (n: number) => console.log('Result is: ', n)

let myFunction: (a: number, b:number) => number;

myFunction = add
console.log(myFunction(5, 5))

myFunction = logResult // ts error
console.log(myFunction(5, 5))
```

- When we want to specify that some variable or parameter should be function we need to describe the function as above
- TypeScript will throw error when assigning logResult to myFunction because it does not contain the exact parameters and the exact return type

Function types and Callbacks

TypeScript

```
function squareAndHandle(n: number, cb: (a: number) => void) {
    const result = n * n
    cb(result)
}

squareAndHandle(9, (data) => {
    console.log(data)
    console.log(data.length) // ts error
    // Property 'length' does not exist on type 'number'
})
```

- Similar as previous definition of Function types, we can also define callback functions.
- In the usage TypeScript will know that data is number and it will allow us to use only properties and methods suitable to numbers

Unknown

TypeScript

```
let input: unknown
let firstName: string

input = 5
input = 'Value'

firstName = input // ts error -> Type 'unknown' is not assignable to type 'string'

if(typeof input === 'string'){
    firstName = input
}
```

- Unknown is a bit more strict than “any”, and it should be used over any but additional checks should be made in order to satisfy typechecks

Never

TypeScript

```
function customError(message: string, code: number): never {
    throw { message, errorCode: code }
}

const result = customError('Some error happened!', 500)
console.log(result) // doesn't log anything
```

- Never is type that is mostly used as return type of functions that will never return anything, not even undefined

TSC compiler options

- As a reminder typescript files are not recognized by browsers and nodejs, they need to be transpiled(translated) in order to be executed.
- We previously installed TypeScript globally using “npm i -g typescript” that gave us access to “tsc” command everywhere
- Than we were able to use “**tsc app.ts**” that will generate app.js file
- In order not to run this command everytime we change something in our .ts file, we can use the --watch parameter: “**tsc app.ts --watch**” or “**tsc app.ts -w**”
- This will detect any changes and it will automatically transpile our ts file

TSC compiler options

- What if we have multiple Typescript files that we want to compile at once?
- Than we need to initialize typescript config file and setup any options that suit our needs
- Using “**tsc --init**” only once per project will do that for us.
- This command in the exact folder where we execute it will produce “tsconfig.json” file
- From now on, TypeScript will read options from tsconfig.json and it will transpile according to those options
- The command for transpiling whole project (meaning every .ts file) is only “**tsc**”
- We can combine this command with watcher: “**tsc --watch**” or “**tsc -w**”

tsconfig.json

- **target**
  - Specifies which version of JavaScript should be the transpiled code
- **lib**
  - The initial types that we have available by default, or we can specify more
- **sourceMap**
  - Enables debugging typescript code in browser instead of the default javascript code
- **outDir**
  - Tells TypeScript where to store the transpiled .js files
- **rootDir**
  - Tells TypeScript from where to transpile the .ts files
- **removeComments**
  - The generated .js files will not contain any comments
- **noEmitOnError**
  - It will not generate .js files if we have errors in our .ts files
- **strict**
  - Multiple options where we loose or tight typescript checks

Exercise I

- Create new empty folder
- Setup typescript project using “tsc --init”
- Create new folder “src”
- All your TypeScript files should be placed inside “src” folder
- Create new folder “build”
- Adjust the tsconfig.json to save and update the js files inside “build” folder
- Run the “tsc --watch” command

Exercise II

- Fetch all albums from https://jsonplaceholder.typicode.com/albums
- In album container render card for each of the albums received from the endpoint

Exercise III

- Add possibility to click on album
- When clicked it should display all photos that are from that album id
- To achieve this use this url: https://jsonplaceholder.typicode.com/albums/[albumId]/photos
- Where [albumId] should be replaced with real albumId